



DR B. DIOP

babacar.diop@ussein.edu.sn

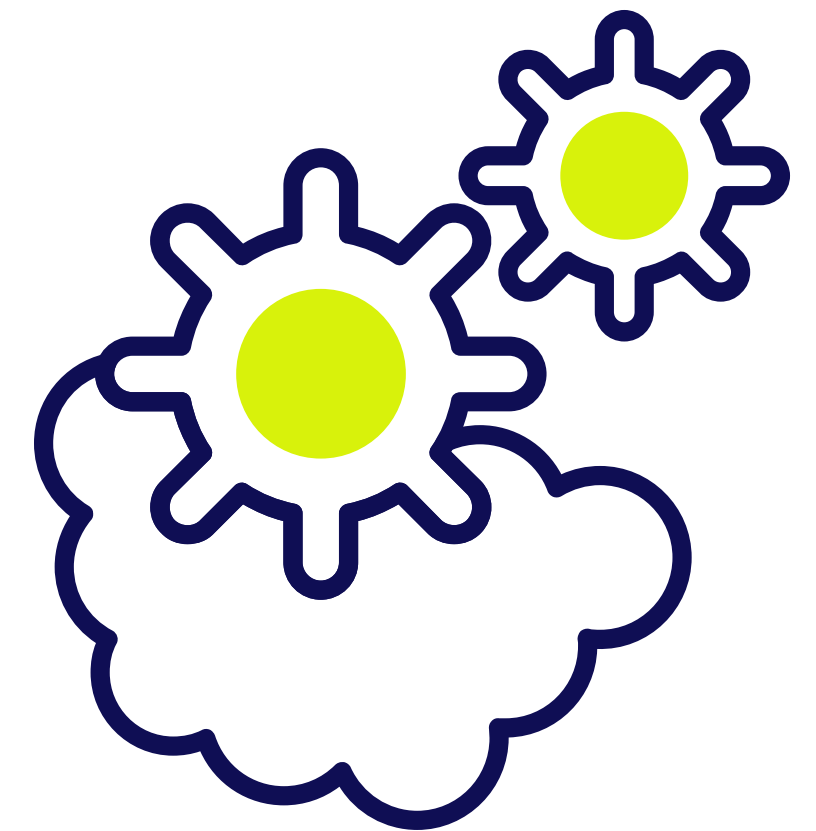
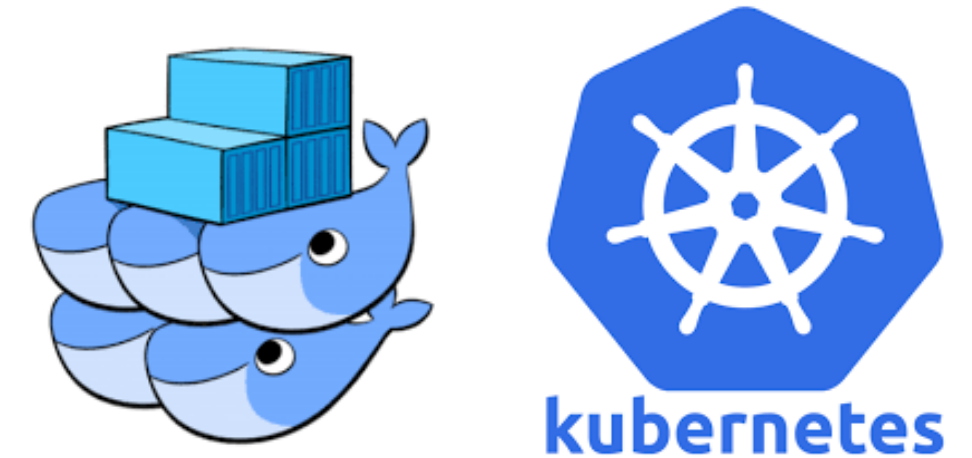
Conteneurisation & orchestration

Kubernetes Orchestration

Ingress

INFO – AGROTIC

Last update: dec. 2025



Rappel - Limites des Services

Services NodePort :

1. N'utilisent que des ports élevés (30000-32767)
2. Requièrent la connaissance des noms ou adresses IP des nœuds

Services LoadBalancer :

1. Permettent un accès externe via un load balancer cloud
2. Mais nécessitent 1 Service = 1 load balancer cloud

Rappel - Limites des Services

Services NodePort :

1. N'utilisent que des ports élevés (30000-32767)
2. Requièrent la connaissance des noms ou adresses IP des nœuds

Services LoadBalancer :

1. Permettent un accès externe via un load balancer cloud
2. Mais nécessitent 1 Service = 1 load balancer cloud

Le problème des Services LoadBalancer

Inconvénients :

- Coûteux : Chaque application exposée a besoin de son propre load balancer
- Limité : Les fournisseurs cloud peuvent imposer des quotas sur le nombre de load balancers
- Exemple : 25 applications → 25 load balancers

Solution : Ingress Kubernetes

Principe et fonctionnement

Principe :

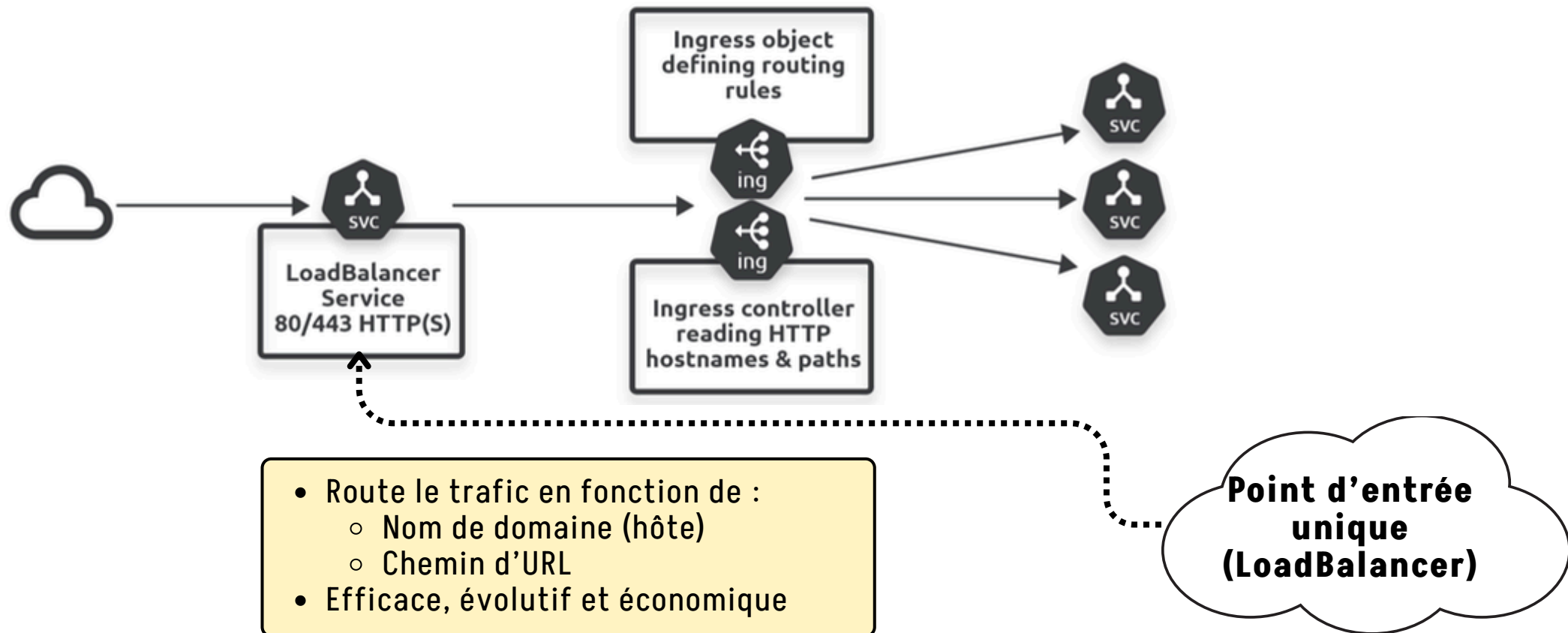
Exposer plusieurs Services via un seul load balancer cloud

Fonctionnement :

- Crée un Service LoadBalancer sur le port 80 (HTTP) ou 443 (HTTPS)
- Utilise du routage par hôte et par chemin d'URL pour diriger le trafic
- Acheminer les requêtes vers le Service backend approprié

Fonctionnement de Ingress Kubernetes

Schéma de fonctionnement



Ingress : Statut et adoption

Architecture Ingress K8s

- **Ingress** est une ressource stable de l'API Kubernetes depuis la version 1.19
- Auparavant en bêta pendant plus de 15 versions (3+ ans en alpha/bêta)
- Popularité croissante des meshes de services – certains chevauchements fonctionnels existent
- À noter : Si vous utilisez un service mesh, **Ingress** peut ne pas être nécessaire

Structure d'Ingress

Architecture Ingress K8s

Ingress repose sur deux composants principaux :

1. Le **contrôleur Ingress** – implémente les règles
 2. La **spécification Ingress** – définit les règles de routage
- Défini dans le sous-groupe API **networking.k8s.io** en tant qu'objet v1
 - **La plupart des clusters Kubernetes ne fournissent pas de contrôleur Ingress par défaut (à installer manuellement)**
 - **Exception** : certains clusters managés (comme GKE) en préinstallent un

Fonctionnement et couche OSI

Architecture Ingress K8s

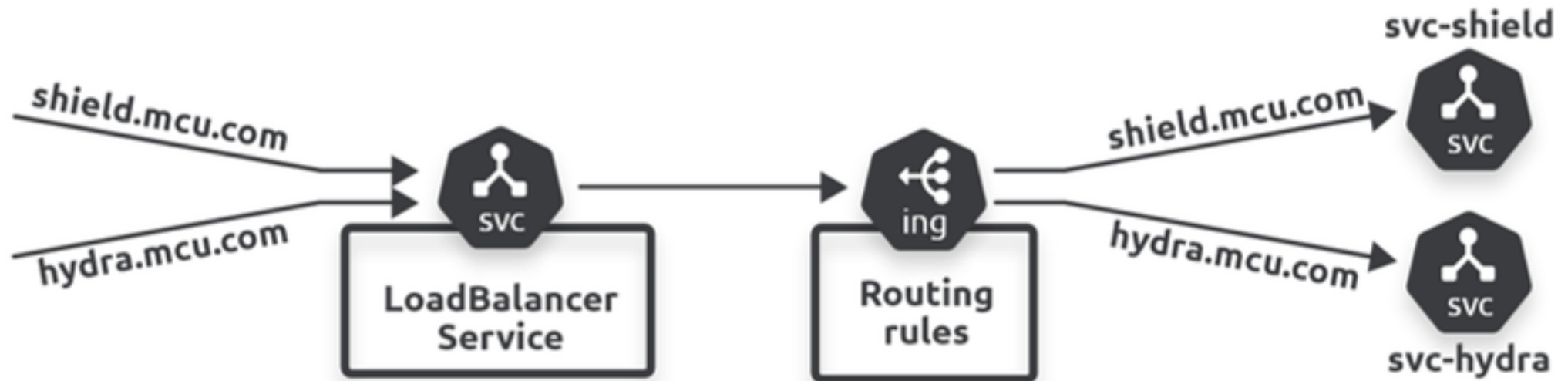
- **Ingress** opère au niveau 7 du modèle OSI (couche application)
- Capable d'inspecter les en-têtes HTTP
- Prend des décisions de routage basées sur :
 - Nom d'hôte (hostname) ◆
 - Chemin d'URL (path) ◆

Type de routage	Requête entrante	Service backend
Par hôte	shield.mcu.com	svc-shield
Par hôte	hydra.mcu.com	svc-hydra
Par chemin	mcu.com/shield	svc-shield
Par chemin	mcu.com/hydra	svc-hydra

Schéma de routage par hôte

Architecture Ingress K8s

- Plusieurs noms d'hôte pointent vers le même load balancer cloud
- L'objet Ingress inspecte l'en-tête Host et route vers le Service ClusterIP approprié
- Nécessite une configuration DNS pour résoudre les noms vers l'IP du LB Ingress



Points clés à retenir

- Ingress permet d'exposer plusieurs Services ClusterIP via un seul load balancer cloud
- Économique et scalable (évite la multiplication des load balancers)
- Le contrôleur Ingress doit généralement être installé séparément
- Fonctionne avec du routage intelligent (hôte + chemin)
- Opère en couche applicative (HTTP)

Prérequis et environnement

- Cluster Kubernetes sur une plateforme cloud avec intégration LoadBalancer
- Exemples basés sur le contrôleur NGINX Ingress
- Clone du repo GitHub du livre nécessaire :

```
git clone https://github.com/nigelpoulton/TheK8sBook.git  
cd ingress
```

Installation du contrôleur NGINX Ingress

- La plupart des clusters ne fournissent pas de contrôleur Ingress par défaut
- GKE est une exception (contrôleur intégré)
- Installation via un fichier YAML depuis le repo Kubernetes :

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v0.44.0/deploy/static/provider/cloud/deploy.yaml
```

Vérification :

```
kubectl get pods -n ingress-nginx
```

Gestion des classes Ingress

- Pourquoi ? Gérer plusieurs contrôleurs Ingress dans un cluster (ex : GKE + NGINX)
- Les IngressClass indiquent quel contrôleur utiliser
- **Exemple de création :**

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: igc-nginx
spec:
  controller: nginx.org/ingress-controller
```

- **Commande :**

```
kubectl apply -f ig-class.yml
```

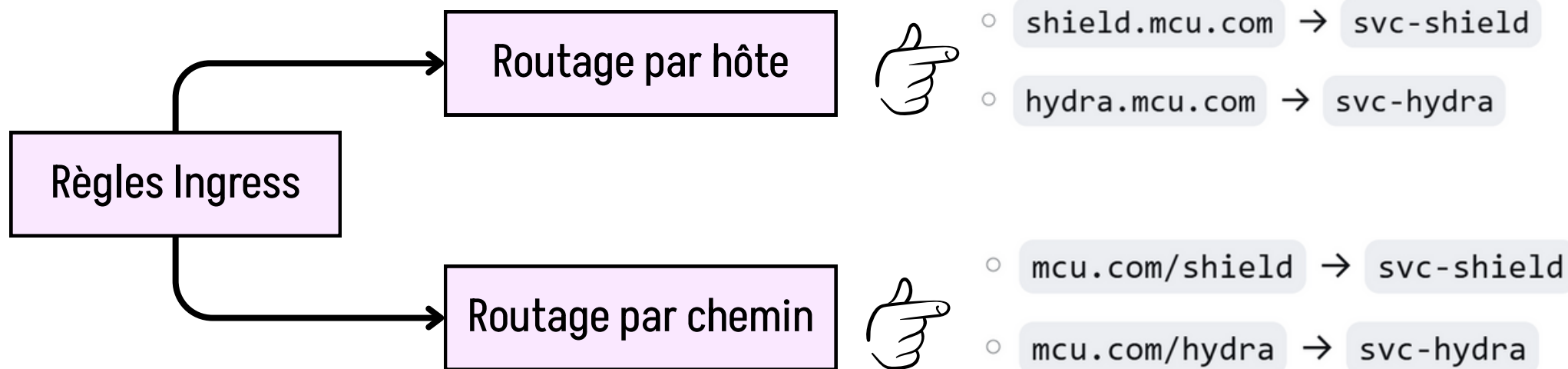
Configuration du routage (hôte et chemin)

Objectif : Exposer deux applications via un seul load balancer

Applications :

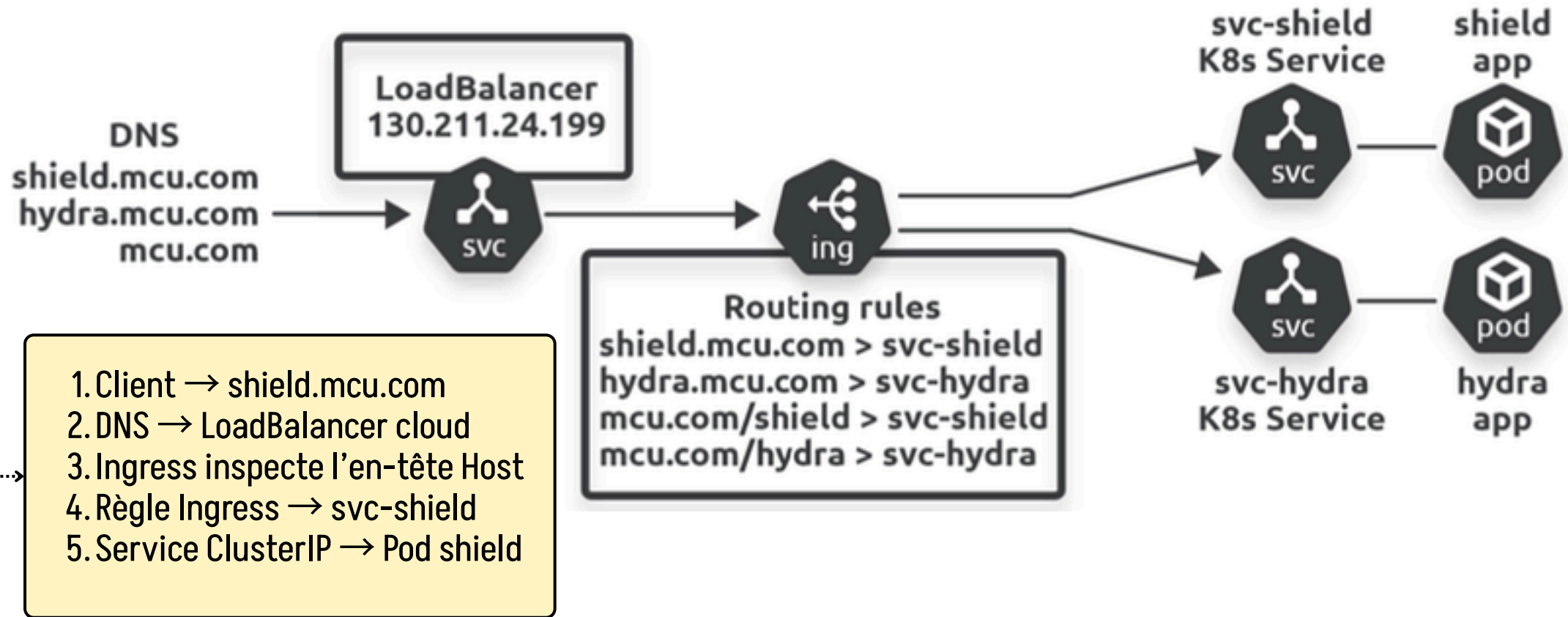
1. `shield` → Service `svc-shield`

2. `hydra` → Service `svc-hydra`



Architecture globale

Flux de trafic (exemple pour shield.mcu.com) :



Déploiement des applications

Fichier app.yml dans le dossier ingress :

- **Deux Pods :**

- shield (port 8080)
- hydra (port 8080)

- **Deux Services ClusterIP :**

- svc-shield
- svc-hydra

- **Commande de déploiement :**

```
kubectl apply -f app.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: svc-shield
spec:
  type: ClusterIP
  ports:
    - port: 8080
      targetPort: 8080
  selector:
    env: shield
---
```

```
apiVersion: v1
kind: Service
metadata:
  name: svc-hydra
spec:
  type: ClusterIP
  ports:
    - port: 8080
      targetPort: 8080
  selector:
    env: hydra
---
```

```
apiVersion: v1
kind: Pod
metadata:
  name: shield
  labels:
    env: shield
spec:
  containers:
    - name: shield-ctr
      image: nigelpoulton/k8sbook:shield-ingress
      ports:
        - containerPort: 8080
      imagePullPolicy: Always
---
```

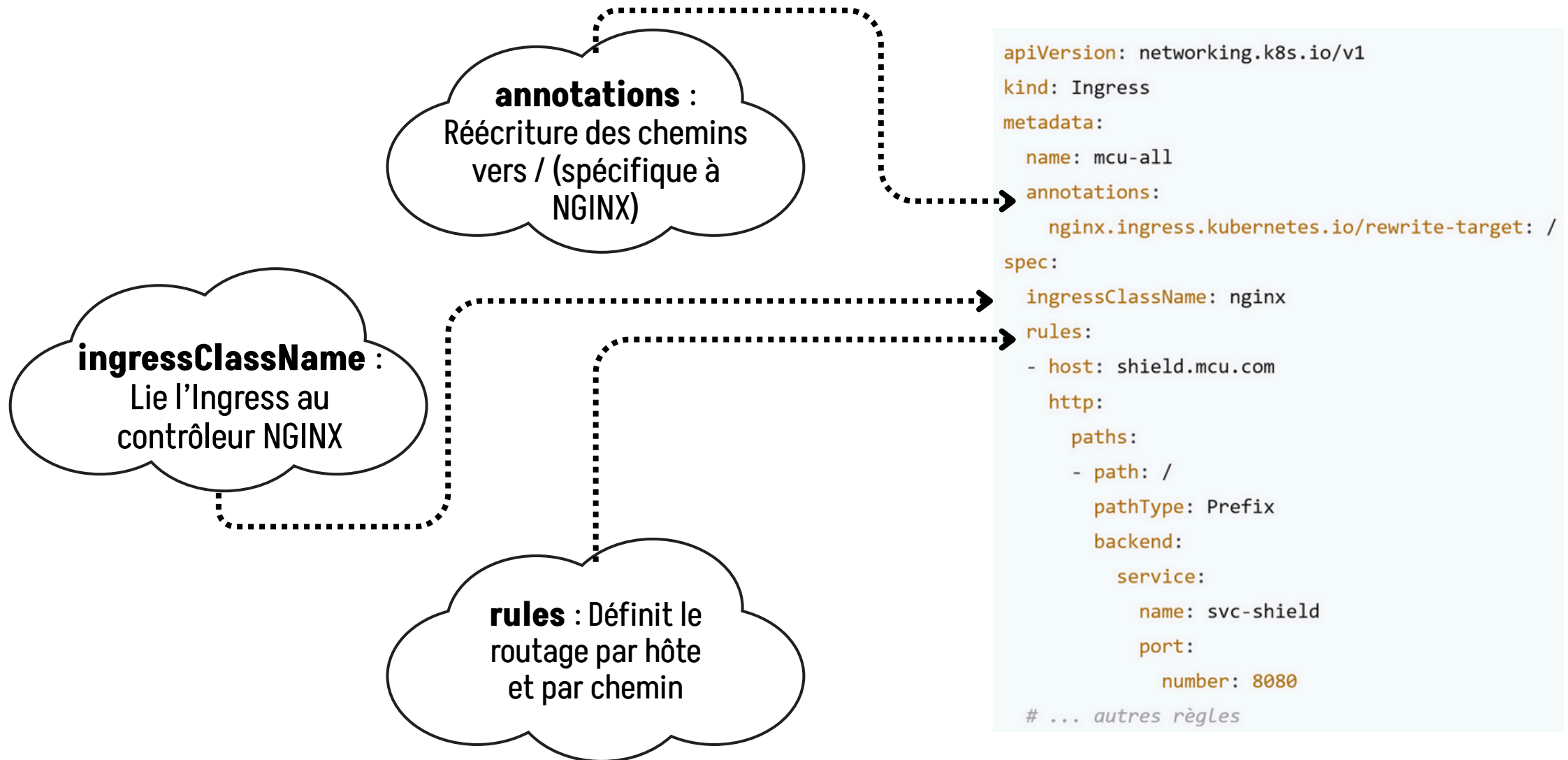
```
apiVersion: v1
kind: Pod
metadata:
  name: hydra
  labels:
    env: hydra
spec:
  containers:
    - name: hydra-ctr
      image: nigelpoulton/k8sbook:hydra-ingress
      ports:
        - containerPort: 8080
      imagePullPolicy: Always
---
```


Objet Ingress - Configuration et déploiement

Présentation du fichier ig-all.yml

- Nom de l'Ingress : **mcu-all**
- Contient **4 règles de routage** :
 1. Routage par hôte : **shield.mcu.com** → **svc-shield**
 2. Routage par hôte : **hydra.mcu.com** → **svc-hydra**
 3. Routage par chemin : **mcu.com/shield** → **svc-shield**
 4. Routage par chemin : **mcu.com/hydra** → **svc-hydra**

Structure de l'objet Ingress



Détail des règles de routage

RÈGLE PAR HÔTE :

```
- host: shield.mcu.com
  http:
    paths:
      - path: /
        backend:
          service:
            name: svc-shield
            port: 8080
```

RÈGLE PAR CHEMIN :

```
- host: mcu.com
  http:
    paths:
      - path: /shield
        backend:
          service:
            name: svc-shield
            port: 8080
```

Déploiement de l'Ingress

Commande :

```
kubectl apply -f ig-all.yml
```

Exemple de sortie :

```
ingress.networking.k8s.io/mcu-all created
```

Inspection de l'objet Ingress

Liste des Ingress :

```
kubectl get ing
```

Exemple de sortie :

NAME	CLASS	HOSTS	ADDRESS	PORTS
mcu-all	nginx	shield.mcu.com,hydra.mcu.com,mcu.com	130.211.24.199	80

- CLASS : Contrôleur utilisé (nginx)
- HOSTS : Noms d'hôte configurés
- ADDRESS : IP publique du load balancer cloud

Description détaillée (kubectl describe)

Sortie simplifiée :

```
Name:          mcu-all
Address:       130.211.24.199
Rules:
  Host          Path  Backends
  ----          -
shield.mcu.com  /      svc-shield:8080
hydra.mcu.com   /      svc-hydra:8080
mcu.com         /shield svc-shield:8080
               /hydra  svc-hydra:8080
Annotations:    nginx.ingress.kubernetes.io/rewrite-target: /
```

- Backend par défaut : Gère le trafic non configuré
- Annotations : Spécifiques au contrôleur (ex : réécriture de chemin)

Configuration DNS temporaire

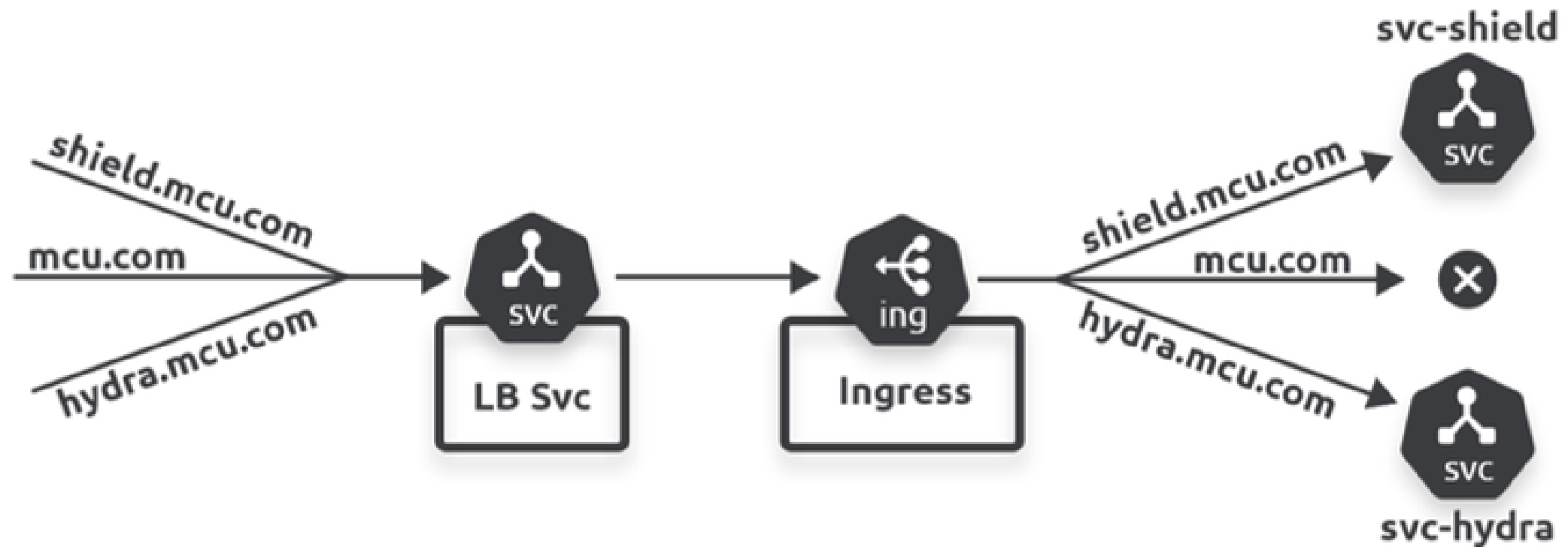
Modification du fichier hosts :

- Mac/Linux : **/etc/hosts**
- Windows : **C:\Windows\System32\drivers\etc\hosts**

Ajoutez (avec votre IP publique) :

```
130.211.24.199 shield.mcu.com  
130.211.24.199 hydra.mcu.com  
130.211.24.199 mcu.com
```

Test des règles Ingress



Test des règles Ingress

URLs à tester :

- **shield.mcu.com** → Routé vers **svc-shield**
- **hydra.mcu.com** → Routé vers **svc-hydra**
- **mcu.com/shield** → Routé vers **svc-shield** (avec réécriture de chemin)
- **mcu.com/hydra** → Routé vers **svc-hydra** (avec réécriture de chemin)

Remarque :

- **mcu.com** seul peut renvoyer une erreur 404 (pas de règle définie)
- La réécriture de chemin peut ne pas fonctionner avec toutes les applications

Etape 1 : supprimer l'objet Ingress

Commande :

```
kubectl delete ingress mcu-all
```

Sortie attendue :

```
ingress.networking.k8s.io "mcu-all" deleted
```

Impact

- Le load balancer cloud associé sera automatiquement supprimé
- Fin des coûts liés au load balancer

Étape 2 - Supprimer les applications et Services

Commande :

```
kubectl delete ingress mcu-all
```

Sortie attendue :

```
service "svc-shield" deleted  
service "svc-hydra" deleted  
pod "shield" deleted  
pod "hydra" deleted
```

Note - Les pods peuvent prendre quelques secondes pour s'arrêter proprement.

Étape 3 - Supprimer le contrôleur NGINX

Supprimer le contrôleur NGINX Ingress (optionnel)

Si vous souhaitez retirer complètement le contrôleur :

1. Supprimer le namespace :

```
kubectl delete namespace ingress-nginx
```

2. Supprimer le clusterrole :

```
kubectl delete clusterrole ingress-nginx
```

3. Supprimer le clusterrolebinding :

```
kubectl delete clusterrolebinding ingress-nginx
```

Étape 4 - Nettoyer le fichier hosts local

Mac/Linux :

```
sudo vi /etc/hosts
```

Windows :

```
C:\Windows\System32\drivers\etc\hosts
```

A supprimer :

```
130.211.24.199 shield.mcu.com
```

```
130.211.24.199 hydra.mcu.com
```

```
130.211.24.199 mcu.com
```

Vérification finale

Vérifiez que tout est bien nettoyé :

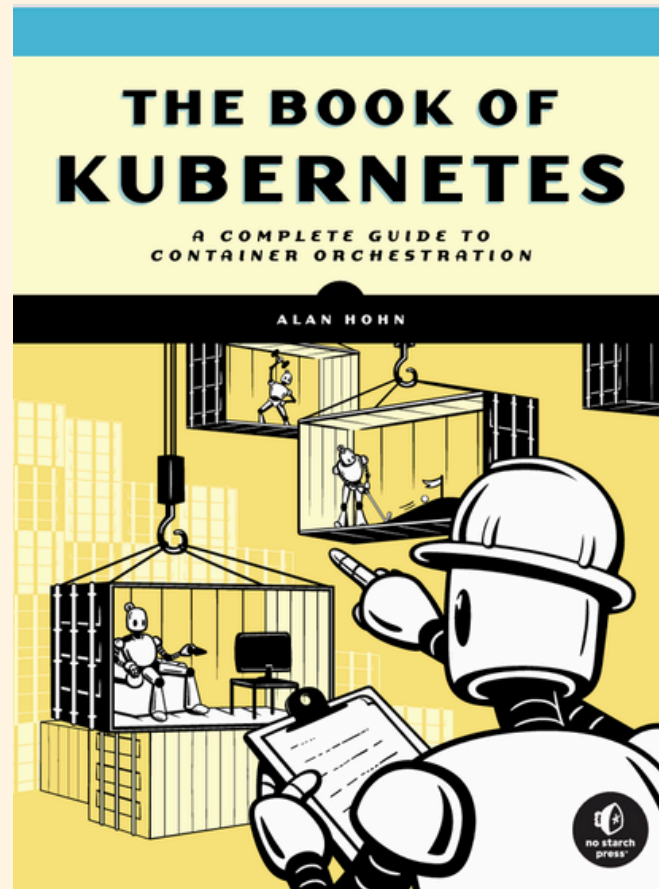
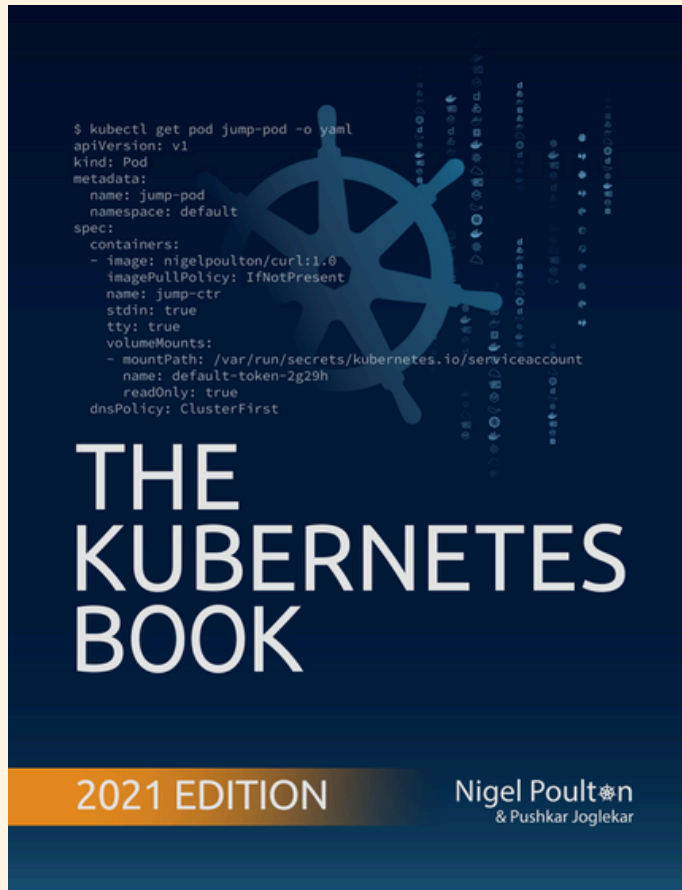
```
kubectl get ingress  
kubectl get pods  
kubectl get services
```

Toutes ces commandes devraient retourner des listes vides (ou sans les ressources Ingress testées).

Bonnes pratiques de nettoyage

- Toujours supprimer les ressources Ingress pour éviter les coûts cloud
- Vérifier la console cloud pour confirmer la suppression du load balancer
- Documenter les étapes pour les futurs tests
- Utiliser des namespaces dédiés pour faciliter le nettoyage

Documentation



Kubernetes Official Website Documentation

Kubernetes Documentation

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications. The open source project is hosted by the Cloud Native...

